

NSTM (Online Neural Space-Time Memory) — 한국어의역본

이 문서는 arXiv:2607.15271 「Online Neural Space Time Memory for Dynamic Novel View Synthesis」 (Elmieh, Tsai, Li, Kaza, Sun, Csapo, Behrouz, Deng, Lombardi, Seitz, Luo (Univ. of Washington / Google)) 의 한국어 의역본입니다. 스타디용이며 공식 번역이 아닙니다. 수식 · 수치 · 그림은 원문 기준이고, 그림은 원저자의 것입니다. 자연스러운 이해를 위해 문장은 의역했습니다. (NM 계열과의 연결 — Behrouz·Li가 공저자이며, 3.3 절의 memory caching 은 arXiv:2602.24281 에서 채택된것입니다. 해석 · 힌트는 easy 판 G11 의 몫.)

초록 (Abstract)

멀티뷰 스트리밍 비디오로부터의 온라인 novel view synthesis(NVS) 는 근본적인 트레이드오프에 직면한다: 일시적으로 가려진 (occluded) 영역을 복원하기 위한 지속적인 장기 기억을 유지하면서, 동시에 엄격한 실시간 제약 아래 동작해야 한다. Test-Time Training(TTT) 은 강력한 메모리 메커니즘을 제공하지만, 표준 모델들은 매스텝 gradient 갱신을 요구해 실시간 처리와 충돌한다. 우리는 NSTM(Neural Space-Time Memory) 을 제안한다 — (1) 메모리 갱신 (memorization) 과 합성 (synthesis) 을 분리 (decouple) 해 갱신은 낮은 빈도로, 합성은 매 프레임 수행하고, (2) 장면의 지속적 내재화 (persistent internalization) 를 강제하는 Memory Loss, (3) 활성 weight 의 파국적 drift 를 억제하는 Memory Caching 전략을 결합한다. 본 방법은 동적 인물 모션 장면에서 실시간 · 최고성과 분 (minute) 스케일 온라인 기억을 시연한다.

1. 서론 (Introduction)

카메라 스트림을 보며 장면을 실시간으로 재구성하는 시스템을 상상하자. 지금 보이는 부분은 현재 관측으로 렌더하면 되지만, 한때 보였다가 지금은 가려진 부분 (예: 사람이 돌아서서 등이 보였다가 다시 정면이 된 경우) 은 기억으로부터 복원해야 한다. 이 기억은 (i) 수십초~1 분 이상 유지되어야 하고, (ii) 실시간 프레임률을 깨지 않아야 한다.

TTT 계열 (fast weight 를 추론 중 gradient 로 갱신하는 신경 메모리) 은 이 목적에 잘 맞는 표현력을 갖지만, 표준 설계는 모든 프레임마다 메모리를 갱신한다 — gradient 스텝의 비용 때문에 실시간에 못 미치고, 긴 시간 동안 반복 갱신하면 fast weight 가 불안정해진다 (drift). NSTM 의 핵심 아이디어는 세 가지다:

1. **Memorization – Synthesis 분리**: 메모리 갱신은 낮은 빈도 (실험에서는 초당 1 회), 읽기 (합성) 는 매 프레임. 그 사이 프레임들은 동일한 메모리를 공유해 읽는다.
2. **Memory Loss($\mathcal{L}_{mem}$)**: 학습 중 현재 입력으로의 우회로를 차단하고 오직 메모리만으로 복원하게 하는 supervision — 정보가 실제로 fast weight 에 내재화 되도록 강제한다.
3. **Memory Caching**: 과거 fast-weight 스냅샷들의 평균 (캐시) 을 유지해, 읽기를 안정화하고 장기 drift 를 억제한다 — 최근 대규모 언어 모델을 위해 제안된 memory caching 전략 (arXiv:2602.24281 계열) 을 채택한 것이다.

2. 관련 연구 (Related Works)

- **Novel view synthesis**: NeRF·3DGS 계열의 per-scene 최적화, 그리고 LVSM 같은 feed-forward(stateless) 모델. stateless 모델은 현재 입력만 보고 최선의 추정을 하므로 가려진 영역에서는 사전 지식 (prior) 에 의존한 추측을 한다.

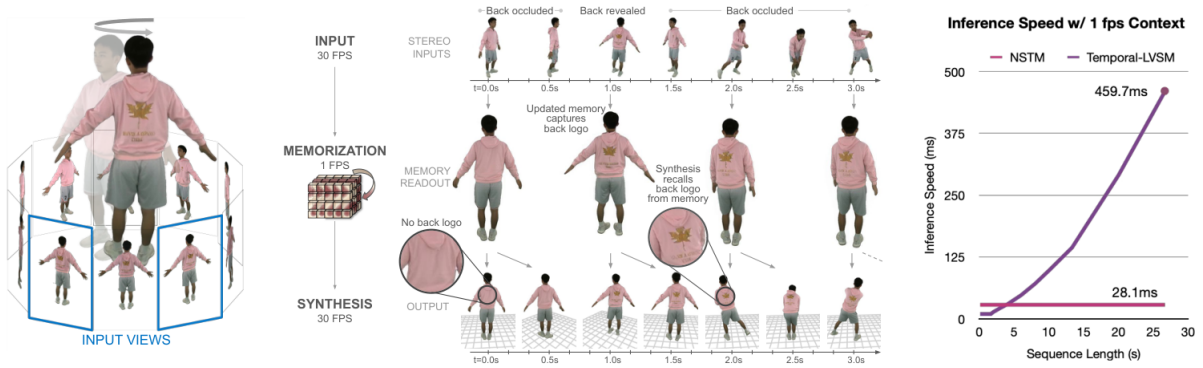


Figure 1. **Neural Space-Time Memory (NSTM)** synthesizes novel views in amortized real-time from 30 FPS multi-view videos, while updating its memory at 1 FPS. **Middle:** NSTM captures the back logo as the subject rotates (see memory readout), faithfully recalling it later from memory given only frontal views. **Right:** While full self-attention in typical feed-forward models (e.g., LVSM [22]) scales quadratically with history sequence length, NSTM scales with $\mathcal{O}(1)$ complexity, achieving amortized real-time inference at 256×256 resolution on one H100 GPU.

그림 1. NSTM 은 amortized 실시간으로 novel view 를 합성한다 —가려진 뒤태를 분 스케일로 기억해 복원. (출처: 원저자, arXiv:2607.15271 Fig.1)

- **Test-Time Training / fast weights:** TTT(Sun et al. 2024), LaCT(Zhang et al. 2025 — large-chunk TTT), DLA/Titans 계열 (Behrouz et al.) — 시퀀스를 fast weight 에압축하는 신경 메모리. 본 논문은 이 계열을 온라인 동적 NVS 에 가져온다.
- **Token 기반 메모리:** 과거를 토큰 버퍼로 유지하는 방식 (Token-Mem baseline) — 길이에 따라 비용이 자라고, 실험에서 정확한 장기 recall 에실패함을 보인다.

3. 방법 (Method)

문제 설정. 시각 t 에 멀티뷰 관측 $O_t = \{(I_{t,i}, P_{t,i})\}$ 가 들어온다 — $I_{t,i}$ 는 RGB 이미지, $P_{t,i} \in SE(3)$ 는 대응 카메라 포즈. O_t 는 (가림 등으로) 참동적 장면의 **불완전한 관측**이므로, NSTM 은 현재 입력 O_t 와누적된 **암묵 메모리** W_{t-1} 양쪽에 조건해 target 이미지를합성한다:

$$I_t^{tgt} = \Phi \left(P_t^{tgt}, O_t; W_{t-1} \right). \quad (1)$$

아키텍처. LVSM 을 따라 입력 RGB 에 per-pixel Plücker ray map 을채널 결합해 linear 층으로 patchify 하여 입력 token 을 만들고, target 카메라의 Plücker ray 도 patchify 해 target token 을 만든다. 이 token 들이 $L = 24$ 개의 **Transformer block** 스택을 통과한다 — 각 block 은 **cross-view attention 모듈**, **TTT memory 모듈**, **feed-forward MLP 모듈**로 구성된다 (그림 7). 마지막에 target token 을 unpatchify 하고 sigmoid 를 적용해 RGB 를 예측한다.

TTT memory 모듈. Test-Time Training/Memorization 은추론 중에 갱신되는 fast weight 집합으로 **압축형 신경 메모리**를정의한다 — 추론 중 **얼어 있는 slow weights**(모델 파라미터) 와대비된다. 본 아키텍처의 메모리는 **bias 없는 SwiGLU-MLP** 다:

$$f_W(x) = (\text{SiLU}(W_1 x) \odot (W_3 x)) W_2. \quad (2)$$

입력 token 에서 key k ·query q ·value v 를 사영하고, 읽기는 $o = f_W(q)$. 갱신은 key 를 value 에 binding 하는 inner loss 에 대한 **단일 gradient** 스텝으로문맥을 기억한다:

$$W' = W - \eta \nabla_W \mathcal{L}_{inner}(f_W(k), v). \quad (3)$$

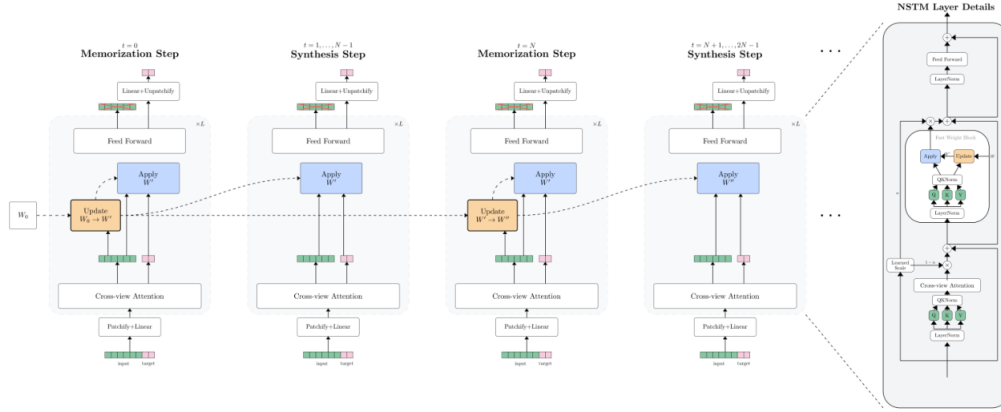


Figure 7. **Online Inference Pipeline and NSTM Layer Design.** **Left:** During inference, we run the memorization step periodically while performing the synthesis step per-frame, which enables us to achieve amortized real-time speed. **Right:** We introduce the NSTM Layer consisting of cross-view attention, a fast-weight block and a feed-forward layer. We also incorporate a learned scale that allows the network to balance the contribution of memory and attention to each layer’s output.

그림 7. 온라인 추론 파이프라인과 NSTM layer 설계. (출처: 원저자, arXiv:2607.15271 Fig.7)

inner loss의 선택이 기존 아키텍처들을 가르는 설계 지점인데, 본 논문은 **L2 inner loss**를 쓴다:

$$\mathcal{L}_{inner} = \|v - f_W(k)\|_2^2. \quad (4)$$

표 2·그림 5에서 보이듯, 일부 선행 모델 (LaCT, DLA)의 **unbounded dot-product loss**는 긴 지평에서 반복 **gradient descent** 후 $f_W(k)$ 의 심각한 크기 팽창 (**magnitude inflation**)을 일으키는 반면, L2 목적은 장기 안정성을 준다. 갱신은 추가로 **gradient**의 **Newton – Schulz** 직교정규화 (Muon)와 **L2 weight normalization**으로 안정화한다.

3.1. Memorization과 Synthesis의 분리 (Decoupled Memorization and Synthesis)

들어오는 모든 프레임마다 기억 (gradient 갱신)을 수행하는 소박한 설계는 비용이 과도하다. NSTM은 두 연산을 분리한다:

- **Memorization step**(낮은 빈도): 현재 입력으로 fast weight W 를 갱신. memory token들은 모듈 안에서 자기들끼리만 **self-attend**하도록 설계된다.
- **Synthesis step**(매 프레임): 갱신 없이 현재 W (정확히는 3.3의 안정화된 읽기 상태)를 읽기만 하며 novel view를 합성. **cross-view attention**이 현재 포즈·변형과 (오래된) 메모리를 정렬한다.

즉 기억을 고치는 빈도와 기억을 쓰는 빈도를 분리했다. 실험설정에서는 memorization을 **초당 1회** 수행하고, 그 사이 약 30개의 synthesis 프레임이 같은 메모리를 읽는다 (4.4 절의 효율 수치 참조).

3.2. 분리된 메모리 감독 (Disentangled Memory Supervision)

fast weight를 TTT loss로 갱신하는 것만으로는 부족하다 — 모델이 **현재 입력을 우회 경로로** 써서 답을 만들 수 있기 때문이다 (그 경우 정보는 메모리에 실제로 저장되지 않는다). NSTM의 학습은 **memorization**과 **synthesis**를 **1:1**로 교대하며, 서로 다른 supervision을 준다:

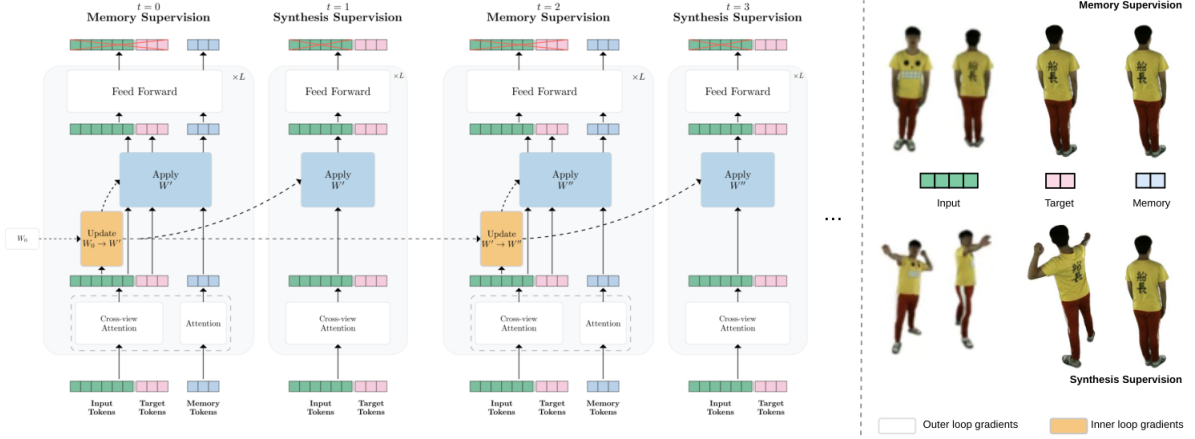


Figure 2. **NSTM Training Scheme.** **Left:** Our alternating training scheme explicitly enforces memorization. In the *Memory Supervision* step, isolated *memory tokens* (target camera rays) perform strict self-attention — sharing weights with the *cross-view attention* layer — without attending to input views. Once the memory is updated by the inputs, these isolated tokens query the memory to reconstruct the target view. This forces a pure memory readout, ensuring context internalization. During *Synthesis Supervision*, we perform self-attention across input and target tokens to capture ongoing motion, and then query the previously updated memory state. **Right:** Visualizations of the inputs, targets, and memory in both steps. Crucially, the historical memory state often exhibits a different pose from the current observation during synthesis. Our *cross-view attention* addresses this deformation, successfully fusing past appearance with current motion.

그림 2. NSTM 학습 스킴 — 교대 학습이 memorization 과 synthesis 를 명시적으로 분리. (출처: 원저자, arXiv:2607.15271 Fig.2)

- 홀수 timestep ($t = 1, 3, \dots$): 통상적인 **synthesis supervision** — 현재 입력과 메모리를 함께 써서 target 을 복원.
- 짝수 timestep: **memory-only supervision** ($\mathcal{L}_{mem}$) — target token 이 현재 입력 token 을 attention 으로 볼 수 없게 차단하고, 오직 fast memory 에 저장된 정보만으로 이미지를 복원하게 한다.

이 보조 손실 $\mathcal{L}_{mem}$ 이 장면의 견고한 기하학적내재화 (robust geometric internalization) 를 강제한다 — “읽을 수 있었던 것” 이 아니라 “파라미터에 실제로 들어간 것” 만 인정하는 검사다. 학습은 update:synthesis 를 1:1 로 번갈아 하지만, 추론에서는 한 번 갱신한 기억을 여러 synthesis 프레임이 공유할 수 있다.

3.3. Memory Caching

주기적 갱신을 순차 적용하면 진화하는 메모리 상태열 $W_0, W_1, W_2, \dots$ 가 생긴다 (갱신 사이의 상태는 동일). 어느 시점 t 에서든 **활성 메모리**는 W_{t-1} 이다 — 스냅샷을 찍은뒤에도 메모리는 리셋되지 않는다. 장기 지평에서의 drift 를 다스리기 위해, **최근 대규모 언어 모델용으로 제안된 memory caching 전략**을 채택한다: 매 K 번째 갱신마다 스냅샷을 저장하고, 과거 스냅샷들의 **running arithmetic mean** 을 재귀적으로 유지한다:

$$\bar{W}_0 = W_0, \quad \bar{W}_n = \frac{n-1}{n} \bar{W}_{n-1} + \frac{1}{n} W_{nK}, \quad \forall n \geq 1.$$

읽기 시점에는 활성 메모리를 그대로 쓰지 않고, **과거 평균과 혼합한 상태**를 읽는다:

$$\hat{W}_{t-1} = \frac{1}{m+1} \left(W_{t-1} + \sum_{i=1}^m W_{iK} \right) = \frac{W_{t-1} + m \bar{W}_m}{m+1}, \quad m = \left\lfloor \frac{t-1}{K} \right\rfloor.$$

쓰기 (갱신) 는 계속 활성 W 에 수행되고, 읽기는 안정화된 $\hat{W}$ 를 사용하며, 과거 평균을 활성 W 에 되쓰지는 않는다. 추가 메모리는 파라미터 크기의 상태 한 세트면 충분하다.

4. 실험 (Experiments)

4.1. 실험 설정

데이터. MVHumanNet++ — 4,500 명 · 9,000 벌 의상에 걸친 60,000 개 멀티뷰 모션 시퀀스 (90%/10% train/test). 넓은 시점 커버리지 · 비제약모션 · **분 단위 시퀀스 길이**가 동적 NVS · 장기 기억 테스트베드로 적합하다. 평가 시퀀스는 60 프레임 (원시 50 초 영상을 1.2 FPS 로 서브샘플) — 분 스케일 평가가 가능하다. 자연 상태에서 완전 회전이 있는 장면은 390 개 중 17 개뿐이라, 두프로토콜을 쓴다.

1. **Memory Stress Test**(강제 가림): $T = 0$ 에 전신 앞뒤를 보여 준뒤, 이후에는 **정면 스테레오만** 입력하고 가려진 **뒷면을 합성**하게 한다. $T = 4, 30, 60$ 프레임에서 기억 감쇠를 정량화.
2. **Memory from Natural Rotations**: 자연 회전이 있는 장면에서의 평가.

훈련. 512×512 에서 다중 프레임 fine-tuning, **$128 \times \text{B200}$ GPU** 사용. **Baselines:** LVSM(stateless), **LaCT-NVS**(large-chunk TTT 를 NVS 에 적용), **Token-Mem**(토큰버퍼 메모리).

4.2. Memory Stress Test 결과

$T = 60$ (약 1 분) 에서도 NSTM 은 $T = 0$ 에 한 번 본 후드무늬 · 헤어스타일을 정확히 recall 하면서 현재 모션과 융합한다. LVSM 은 **보이지 않는 영역에서 stateless 추측** (prior) 으로 후퇴하고, **LaCT-NVS** 는 **out-of-distribution 메모리 drift** 를 겪으며, **Token-Mem** 은 **정확한 뒷면 recall 에 실패**한다. 단기 ($T = 4$) 에는 대안들도 그럭저럭이지만, 장기로 갈수록 명시적 · 안정화된 메모리의 이점이 커진다 — baseline 들이 drift 하거나 stateless prior 로 붕괴하는 동안 NSTM 은 높은 충실도를 유지한다 (그림 3·4).

4.3. Ablation (표 2 요지)

구성	PSNR
LaCT-NVS baseline (dot-product loss)	22.32
LaCT-NVS + L2 loss	27.77
LaCT-NVS + L2 + Memory Caching	20.25
Ours without $\mathcal{L}_{mem}$	28.23
Ours without Memory Caching	29.24
Full NSTM	30.09

세 가지 관찰이 결정적이다.

1. **L2 inner loss**: dot-product loss(22.32) 를 L2 로 바꾸는 것만으로 27.77 — unbounded loss 의 크기 팽창이 장기불안정의 큰 원인.
2. **LaCT 에 memory caching 을 더하면 오히려 붕괴 (20.25)**: LaCT 의 fast weight 에는 물체의 identity/외형과 함께 **현재포즈 · 변형 · 시점 종속 상태**가 뒤섞여 있어, 시간에 걸쳐 파라미터를 평균하면 서로 다른 포즈 · 변형이 겹쳐 심각한 성능 저하를 낳는다.
3. **NSTM 에서는 caching 이 이득 (29.24 → 30.09)**: NSTM 은 **fast memory** 에는 비교적 불변인정보를, **현재 포즈 · 변형**과의 정렬은 **cross-view attention** 에 맡기도록 역할을 분리했기 때문에, 스냅샷 평균이 의미를 유지한다. 즉 **memorization – synthesis 분리 (cross-view attention 포함)** 가 **memory caching** 이 작동하기 위한 전제다. $\mathcal{L}_{mem}$ 을 빼면 28.23 으로 떨어진다 — 메모리에 실제로 넣도록 압박하지 않으면 내재화가 약해진다.

LVSM	29.13	0.9666	0.0258	0.1040	19.55	0.5790	28.98	0.9656	0.0263
LaCT-NVS	30.69	0.9741	0.0196	0.0822	21.72	0.7110	30.38	0.9708	0.0216
Token-Mem	29.36	0.9674	0.0249	0.1009	20.40	0.6409	28.36	0.9621	0.0290
Ours	30.43	0.9731	0.0203	0.0851	21.46	0.6971	30.03	0.9700	0.0225

Table 1. **Memory Stress Test Quantitative Comparisons.** The best, second best, and third best results are highlighted in pink, yellow, and light blue, respectively. NSTM maintains high-fidelity recall over time while baseline models suffer from drift or collapse to stateless prior over time. Please refer to our supplementary website for comprehensive

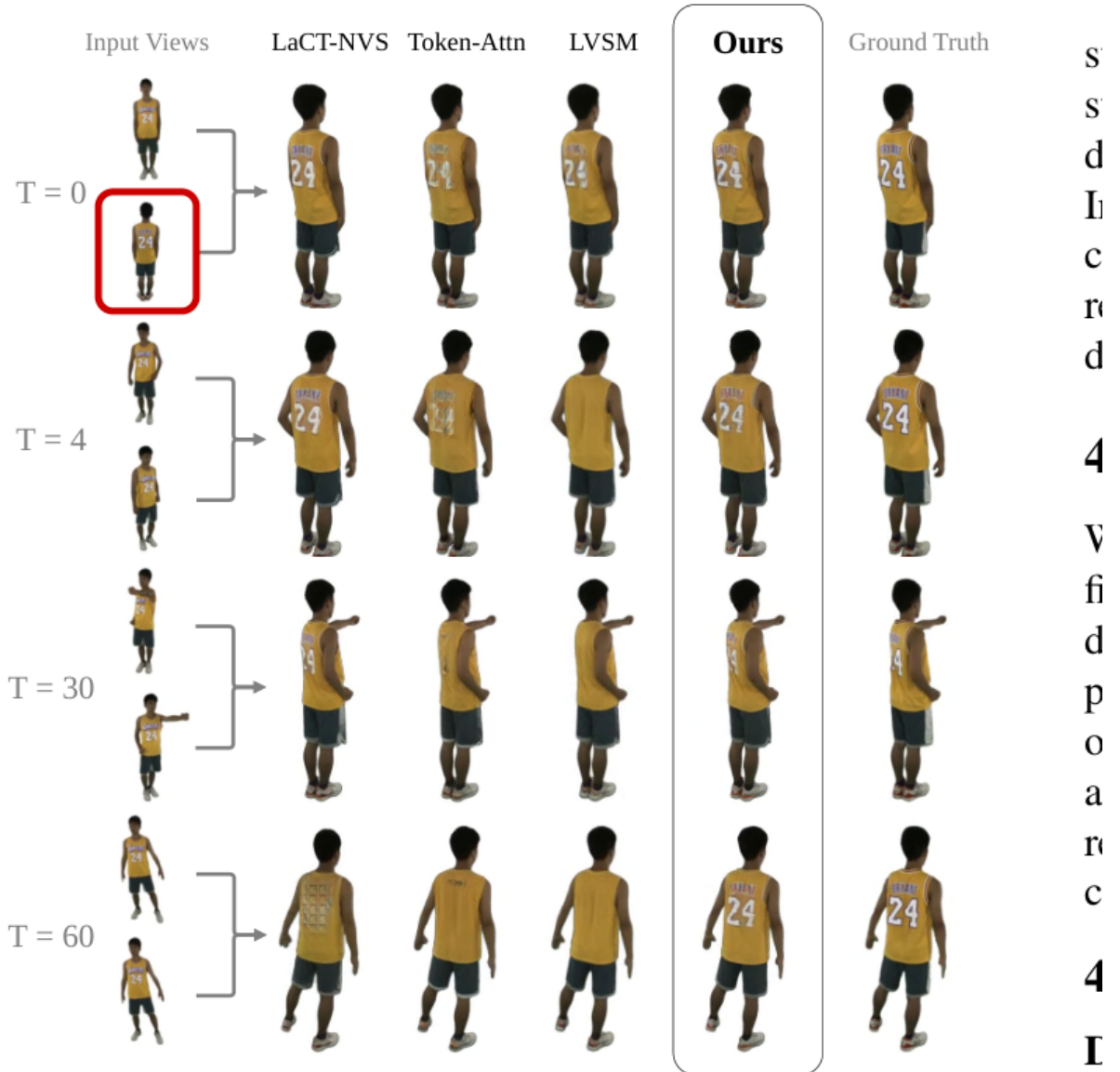


Figure 4. **Memory Stress Test Over Time.** We show the back view at T=0. For subsequent timestamps, we show only frontal views and task the model with synthesizing the occluded back. NSTM maintains high-fidelity recall over time while baseline models suffer from drift or collapse to stateless prior over time.

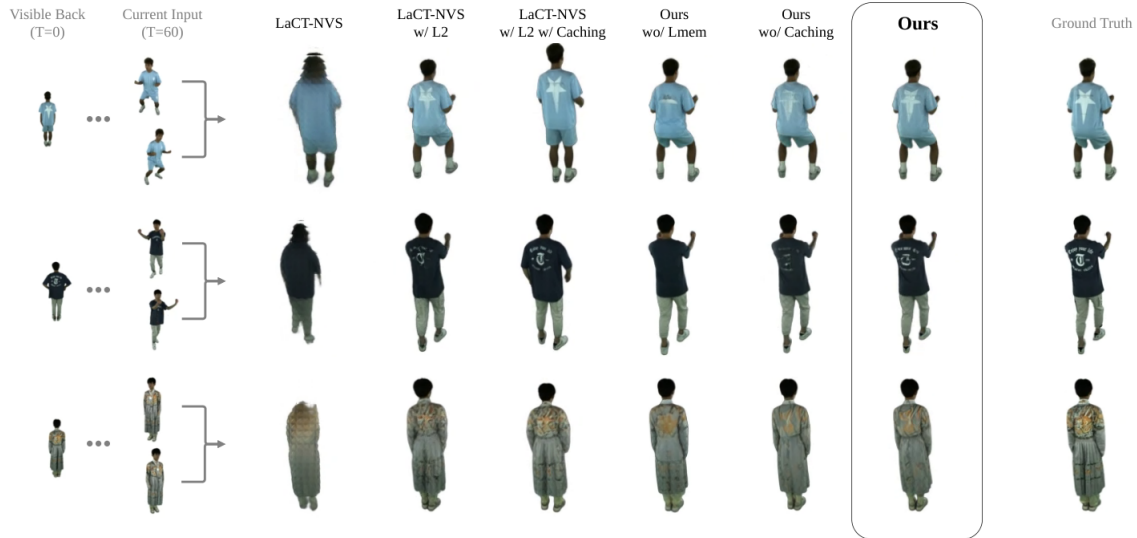


Figure 5. **Qualitative ablation study.** At $T = 60$, each model synthesizes the back view that has been occluded after $T = 0$. **LaCT-NVS** demonstrates the instability of the inner dot-product loss. In contrast, **LaCT-NVS w/ L_2** greatly improves stability through our L_2 inner loss choice, though it still suffers from some drift. **LaCT-NVS w/ L_2 w/ Caching** freezes into an averaged past pose, demonstrating that our memorization-synthesis decoupling with cross-view attention is essential for effective memory caching. Dropping $\mathcal{L}_{mem}$ from our method collapses recall, validating our disentangled memory supervision design (Sec. 3.2). Furthermore, dropping memory caching reduces the recall horizon. Only the full model (**Ours**) provides minute-scale recall of the historical back view while capturing current motion. See our supplementary website for the video results.

그림 5. 정성 ablation — $T = 60$ 에서 각 구성의 뒷면 합성 비교. (출처: 원저자, arXiv:2607.15271 Fig.5)

4.4. 실시간 추론효율

단일 H100 기준: **memorization step(갱신+적용) 58.14ms, synthesis step(적용만) 27.01ms**. 메모리 갱신을 초당 1 회로 두고 그 사이 synthesis 프레임들이 같은 메모리를 읽으면, **amortized 약 28.1ms/frame** — 실시간에 해당한다. (memorization 17.19 FPS, synthesis 37.02 FPS.)

5. 한계 (Limitations)

- 갱신 주기와 놓친 사건: 주기적 갱신은 memorization 스텝사이에만 나타난 사건을 놓칠 수 있다 (그림 8 왼쪽).
- **Primacy bias**: 오래된 스냅샷의 균일 평균은 초기 기억을과대 대표할 수 있어, 나중에 드러난 정보가 충분히 반영되지 않을 수 있다.
- 평가 규모는 분 스케일 (60 timestep 수준) 이다 —수일 · 수개월의 lifelong 학습 증거는 아니다.

결론

NSTM 은 온라인 동적 NVS 를 위해 (1) memorization – synthesis 분리, (2) memory-only supervision($\mathcal{L}_{mem}$), (3) memory caching(스냅샷 평균 읽기) 을 결합해, 실시간 프레임률과 분 스케일 장기 기억을 동시에 달성했다. 핵심 실증은 다음이다: 읽기와쓰기의 빈도를 분리하고, **consolidation 대상(fast memory)** 에서상황 종속 변동성을 분리해 내면 (cross-view attention 으로), 단순한 스냅샷산술 평균조차 강력한 장기 안정화 장치가 된다.



Figure 8. **Limitations.** **Left: Update Frequency and Missed Events.** Our periodic updates may miss events between memorization steps. The pattern shown at $t = 1$ was presented only at this non-update timestep and is therefore not recoverable at $t = 60$. **Right: Memory Primacy Bias** Our method favors memorization during early steps. Here the back is revealed after 21 updates (at $t = 42$) and our recovery at $t = 60$ is only partial.

그림 8. 한계 — 갱신 빈도와 놓친 사건, primacy bias. (출처: 원저자, arXiv:2607.15271 Fig.8)